

Projekt Zaliczeniowy
Elementy Sztucznej Inteligencji

SPRAWOZDANIE

Sztuczne sieci neuronowe i Uczenie Maszynowe

Przewidywanie zdrowia psychicznego i długości snu

Autor: Paulina Kania
Kierunek: Informatyka i Ekonometria
Zbiór Danych: Impact of Screen Time on Mental Health

Spis treści

1. CZĘŚĆ I — Sztuczne Sieci Neuronowe	3
1.1. Cel projektu	3
1.1.1. Model Klasyfikacyjny - przewidywanie stanu zdrowia psychicznego ...	3
1.1.2. Model regresyjny — przewidywanie długości snu	3
1.2. Przegląd Literatury	3
1.3. Wstęp do Analizy	4
1.3.1. Przygotowanie danych (Preprocessing)	4
1.3.2. Procedura testowa	4
1.4. Analiza Modelu Klasyfikacyjnego	5
1.4.1. Struktura Sieci Neuronowej	5
1.4.2. Parametry bazowe (Baseline)	6
1.4.3. Zestawienie testowanych parametrów i ich wartości	6
1.4.4. Wnioski końcowe – Klasyfikacja	10
1.5. Analiza Modelu Regresyjnego	12
1.5.1. Struktura Sieci Neuronowej	12
1.5.2. Parametry bazowe (Baseline)	13
1.5.3. Zestawienie testowanych parametrów i ich wartości	13
1.5.4. Wnioski końcowe – Regresja	17
2. CZĘŚĆ II — Uczenie Maszynowe	18
2.1. Cel Projektu	18
2.2. Opis wybranych metod uczenia maszynowego	18
2.2.1. K-Najbliższych Sąsiadów (K-Nearest Neighbors)	18
2.2.2. Las Losowy (Random Forest)	18
2.2.3. Multi-Layer Perceptron (MLP / Sieci Neuronowe)	18
2.2.4. SVM	19
2.3. Analiza Modeli Klasyfikacyjnych	19
2.3.1. k-Najbliższych sąsiadów	19
2.3.2. Las Losowy	22
2.3.3. MLP Classifier	24
2.3.4. SVM	27
2.4. Analiza Modeli Regresyjnych	29
2.4.1. k-Najbliższych sąsiadów	29
2.4.2. Las Losowy	32
2.4.3. MLP Classifier	34
2.4.4. SVM	37
2.5. Podsumowanie i Wnioski Końcowe	39
2.5.1. Ranking skuteczności modeli	39
2.5.2. Kluczowe obserwacje	39

1. CZĘŚĆ I — Sztuczne Sieci Neuronowe

1.1. Cel projektu

1.1.1. Model Klasyfikacyjny - przewidywanie stanu zdrowia psychicznego

Celem tej części projektu to automatyczna ocena czy ktoś choruje na podstawie wymienionych poniżej. Dana osoba jest kwalifikowana jako osoba z problemami zdrowotnymi, jeśli spełniony jest jeden z tych warunków:

- `mental_health_score` < 0.4
- `stress_level` > 0.7
- `weekly_anxiety_score` > 0.8

Wynik 1 oznacza, że osoba ma problemy psychiczne spowodowane nadużywaniem urządzeń elektrycznych lub złych nawyków, 0 – jest zdrowa. Jest to bardzo ogólny system zero jedynkowy przy którym dużo osób nawet przy normalnych ilościach snu okazują się być niezdrowe.

1.1.2. Model regresyjny — przewidywanie długości snu

Przewidujemy ilość godzin snu na podstawie zmiennych takich jak: wiek, płeć, dzienny czas ekranowy, dzienny czas używania telefonu, dzienny czas używania laptopa, dzienny czas używania tableta, dzienny czas oglądania telewizji, dzienny czas korzystania z social mediów, dzienny czas związany z pracą, godziny poświęcone na rozrywkę, godziny poświęcone na granie, jakość snu, ocena nastroju, poziom stresu, godziny aktywności fizycznej w tygodniu, typ miejsca zamieszkania, ocena zdrowia psychicznego, używanie aplikacji wellness, czy dana osoba zdrowo się odżywia, spożycie kofeiny w miligramach na dzień, tygodniowy wynik lęku, minuty uważności na dzień.

1.2. Przegląd Literatury

Ze względu na to, że użyty w projekcie dataset jest nowy, na stronie kaggle.com (z której pochodzi dataset) nie ma opracowań, które poruszały by ten sam problem co projekt. Inni użytkownicy najczęściej przewidują `mental_health_score`, jeśli chodzi o model regresyjny (my przewidujemy ilość godzin snu). Jeśli chodzi o model klasyfikacyjny, są dwie prace poruszające ten temat, lecz jedna zawiera trzy kategorie, które przewiduje (projekt zawiera dwie), a druga dzieli zmienną na podstawie mediany z `mental_health_score`, przy czym my wykorzystujemy do tego dane mówiące również o poziomie stresu i tygodniowego wyniku niepokoju. Ponieważ żadna z dostępnych prac nie rozpatruje identycznego problemu, niemożliwe jest bezpośrednie zestawienie uzyskanych wyników. W szerszym

ujęciu zagadnienia, badania nad związkiem czasu ekranowego ze snem (np. Cain & Gradisar, 2010; Chang et al., 2014) wskazują na słabą do umiarkowanej korelację ($r = 0.2-0.3$), co tłumaczy, dlaczego modele operujące na prawdziwych danych osiągają znacząco gorsze wyniki niż obserwowane w niniejszym projekcie.

1.3. Wstęp do Analizy

1.3.1. Przygotowanie danych (Preprocessing)

Przed podaniem danych do sieci neuronowych wykonałam następujące operacje:

- Usunięcie kolumny identyfikacyjnej `user_id`, która nie niesie żadnej wartości badawczej.
- Zastosowanie One-Hot Encoding metodą `pd.get_dummies` na kolumnach tekstowych (`gender`, `location_type`), co przekształciło je w czytelne dla sieci kolumny binarne.
- Poddanie wszystkich cech ciągłych standaryzacji.

1.3.2. Procedura testowa

Aby wyniki były wiarygodne i odporne na błąd wynikający z losowej inicjalizacji wag, zastosowałam procedurę dziesięciokrotnego powtórzenia dla każdej testowanej wartości.

- Dla każdego badanego parametru model był trenowany i oceniany 10 razy.
- **Klasyfikacja:** Po każdej serii obliczano średnią dokładność (Accuracy) zarówno dla zbioru treningowego, jak i testowego, który pokazuje procent poprawnych wyników sieci
- **Regresja:** Po każdej serii obliczano średni błąd bezwzględny (MAE) zarówno dla zbioru treningowego, jak i testowego, który pokazuje procent godziny różnicy wyniku sieci a faktycznych danych.

Pozwoliło to na wyznaczenie stabilnych trendów i wyeliminowanie wyników przypadkowych.

1.4. Analiza Modelu Klasyfikacyjnego

1.4.1. Struktura Sieci Neuronowej

Zgodnie z wymaganiami projektu zaimplementowałam sieć typu Feedforward Neural Network od podstaw, przy użyciu jedynie biblioteki *NumPy*.

Architektura sieci składa się z trzech głównych warstw:

- **Warstwa wejściowa:** Przyjmuje zestaw 27 przefiltrowanych i zstandaryzowanych cech opisujących profil cyfrowy oraz fizyczny użytkownika.
- **Warstwa ukryta (32 neuronów):** Odpowiada za wychwytywanie prostych korelacji między zmiennymi. Jako funkcję aktywacji wykorzystano ReLU (Rectified Linear Unit), która zeruje wartości ujemne, przyspieszając proces uczenia.
- **Warstwa wyjściowa (1 neuron):** Zakończeniem sieci jest pojedynczy neuron z funkcją aktywacji Sigmoid. Skaluje ona wynik końcowy do przedziału $(0, 1)$, co pozwala interpretować sygnał wyjściowy bezpośrednio jako prawdopodobieństwo przynależności do klasy „niezdrowej”. Próg odcięcia decyzyjnego ustalono na poziomie 0.5 dla klasyfikacji binarnej.

Dodatkowo zostały zaimplementowane:

- **Funkcja straty (Loss Function)** - Do oceny jakości predykcji wykorzystałam funkcję Binary Cross-Entropy (Log Loss). Mierzy ona odległość między przewidzianym prawdopodobieństwem a realną etykietą (0 lub 1). Im silniej model „myli się” w swoich pewnych predykcjach, tym wyższą karę punktową otrzymuje, co wymusza korektę wag.
- **Optymalizacja wag (SGD z mechanizmem Momentum)** - Uczenie sieci odbywa się za pomocą stochastycznego spadku wzdłuż gradientu (SGD) w trybie mini-batchy (po 16 próbek). Aby proces uczenia nie utknął w lokalnych minimach i przebiegał stabilniej, podstawowy algorytm SGD wzbogacono o mechanizm Momentum. Działa on analogicznie do kuli toczącej się ze zbocza – algorytm „pamięta” kierunek poprzednich poprawek wag i dzięki zyskanemu pędowi wygładza skoki matematyczne, uśredniając zmiany gradientów w małych batchach. Dodatkowo wdrożyłam tłumienie współczynnika uczenia (learning rate).

1.4.2. Parametry bazowe (Baseline)

Architektura:	[28, 32, 1]
Funkcja aktywacji	ReLU
Współczynnik uczenia	0.005
Rozmiar batchów	32
Liczba iteracji	500
Seperacja danych	1600 próbek treningowych / 400 testowych

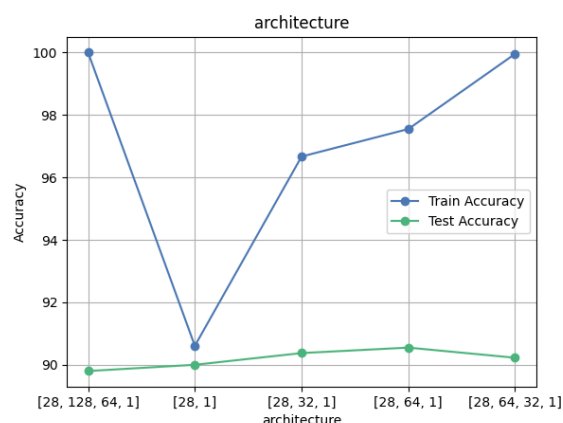
1.4.3. Zestawienie testowanych parametrów i ich wartości

Architektura:	[28,1], [28,32,1], [28,64,1], [28,64,32,1], [28,128,64,1]	Sprawdzenie wpływu głębokości sieci na wykrywanie złożonych zależności.
Funkcja aktywacji	ReLU, Tanh, Sigmoid, Leaky ReLU	Ocena szybkości uczenia i unikania problemu „znikania gradientu”.
Współczynnik uczenia	0.01, 0.005, 0.001, 0.0005	Znalezienie balansu między szybkością a stabilnością zbieżności modelu.
Rozmiar batchów	16, 32, 64, 128	Zbadanie wpływu rozmiaru próbki na stabilność gradientu i czas obliczeń.
Seperacja Danych	1000, 1200, 1500, 1600, 1800	Analiza wpływu ilości danych treningowych na jakość predykcji.
Iteracje	200, 500, 1000, 2000	Wyznaczenie momentu, w którym sieć przestaje się uczyć lub zaczyna się przeuczać.

1.4.3.1. Architektura Sieci

Parameter	Train Accuracy	Test Accuracy
[28, 1]	90.6063	90
[28, 32, 1]	96.6688	90.375
[28, 64, 1]	97.55	90.55
[28, 64, 32, 1]	99.9563	90.225
[28, 128, 64, 1]	100	89.8

Najprostszy wariant bez żadnej warstwy ukrytej czyli [28, 1] wypadł najslabiej, bo przy skuteczności 90% na teście widać, że model jest po prostu zbyt prymitywny, by zrozumieć trudniejsze zależności w danych dotyczących zdrowia psychicznego. Złoty środek wypadłw architekturze [28, 64, 1], która osiągnęła najlepszy wynik testowy na poziomie 90.55%. Dodanie tej jednej warstwy ukrytej dało sieci umiejętność sprawnie łączyć fakty.

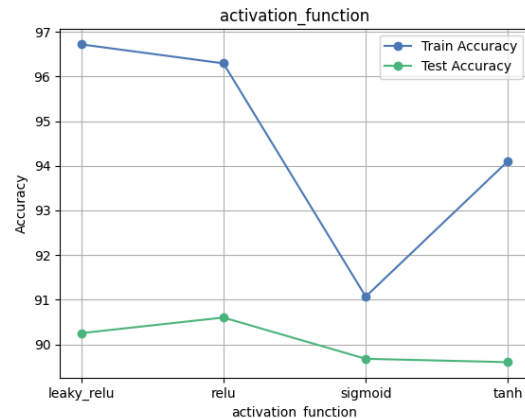


Z kolei każda próba dalszego komplikowania struktury kończyła się klasycznym overfittingiem. Choć na zbiorze treningowym modele te wbijały idealne blisko 100%, to na zbiorze testowym wyniki od razu leciały w dół. To jasny sygnał, że dla tego konkretnego problemu jedna warstwa ukryta to wystarczająca ilość. Każda kolejna sprawia tylko, że sieć zamiast szukać reguł, zaczyna zapamiętywać konkretne przypadki.

1.4.3.2. Funkcja Aktywacji

Parameter	Train Accuracy	Test Accuracy
relu	96.2938	90.6
tanh	94.1063	89.6
sigmoid	91.0688	89.675
leaky_relu	96.7188	90.25

Najlepsze wyniki testowe osiągnęłam przy użyciu funkcji relu (90.6%) oraz jej wariantu leaky_relu (90.25%). Obie te funkcje radzą sobie lepiej z unikaniem problemu zanikającego gradientu. Funkcje sigmoidalne, czyli sigmoid oraz tanh, wypadły nieco słabiej, osiągając na teście wyniki w okolicach 89.6%. W przypadku sigmoidy widać wyraźnie mniejszą zdolność do dopasowania się do danych treningowych (91.06%), co wynika z jej tendencji do nasycania się.

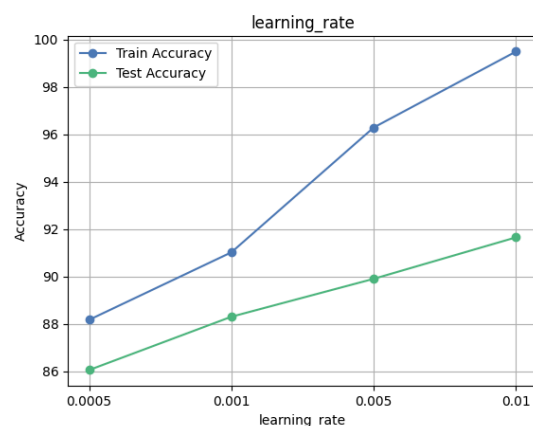


Ostatecznie potwierdza to, że dla tego problemu klasyfikacji najlepiej sprawdza się ReLU, które pozwalają sieci na bardziej agresywną i skuteczną ekstrakcję kluczowych danych.

1.4.3.3. Współczynnik Uczenia

Parameter	Train Accuracy	Test Accuracy
0.01	99.4938	91.65
0.005	96.2875	89.9
0.001	91.0188	88.3
0.0005	88.1688	86.05

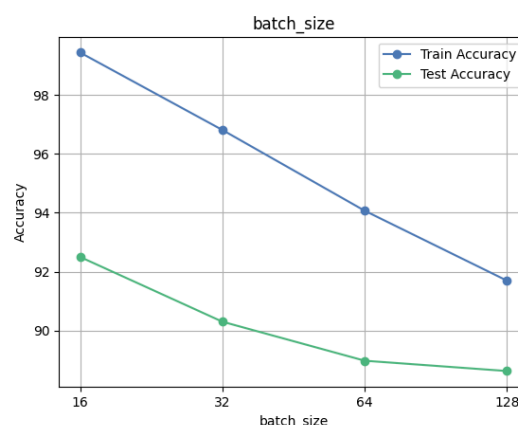
Wyższy learning rate przyspiesza osiągnięcie wysokiej precyzji w wykrywaniu depresji. Najlepsze wyniki uzyskałam dla wartości 0.01, gdzie dokładność testowa osiągnęła szczytowe 91.65%, a model niemal idealnie dopasował się do danych treningowych (99.49%). Wraz ze zmniejszaniem kroku uczenia (0.005, 0.001, aż do 0.0005), skuteczność sukcesywnie spadała, co sugeruje, że przy zbyt małych wartościach sieć potrzebowałaby znacznie więcej epok.



1.4.3.4. Rozmiar batcha

Parameter	Train Accuracy	Test Accuracy
16	99.4438	92.5
32	96.8125	90.3
64	94.0688	88.975
128	91.6938	88.625

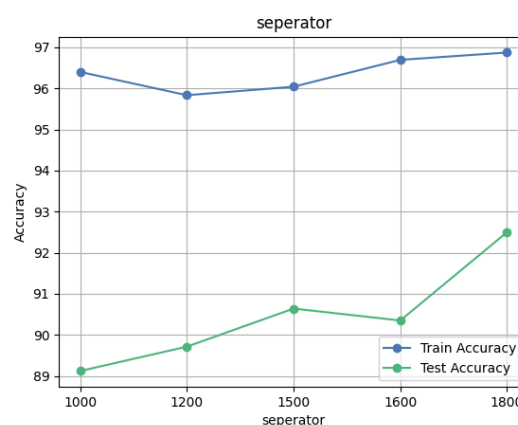
Dla batch size = 16 zostały osiągnięte najlepsze wyniki, gdzie dokładność testowa wyniosła 92.5%, a model niemal idealnie wykrywał wzorce w zbiorze treningowym. Wraz ze zwiększaniem rozmiaru batcha (32, 64, 128), skuteczność na obu zbiorach systematycznie spadała. Wynika to z faktu, że mniejsze batche wprowadzają korzystny „szum” do procesu optymalizacji. Większe partie danych, mimo że stabilizują gradient, w tym przypadku doprowadziły do zbyt dużego uproszczenia nauki i słabszych wyników końcowych.



1.4.3.5. Seperacja Danych

Parameter	Train Accuracy	Test Accuracy
1000	96.4	89.12
1200	95.8333	89.7125
1500	96.04	90.64
1600	96.6938	90.35
1800	96.8722	92.5

Badanie wpływu separacji danych potwierdza złotą zasadę uczenia maszynowego: im więcej wysokiej jakości danych dostarczymy do fazy treningu, tym lepszą precyzję uzyskamy na teście. Najlepszy wynik (92.5% na zbiorze testowym) osiągnęłam przy ustawieniu separatora na 1800, co oznacza, że 90% danych (1800 próbek) trafiło do nauki, a 10% (200 próbek) do sprawdzenia modelu.



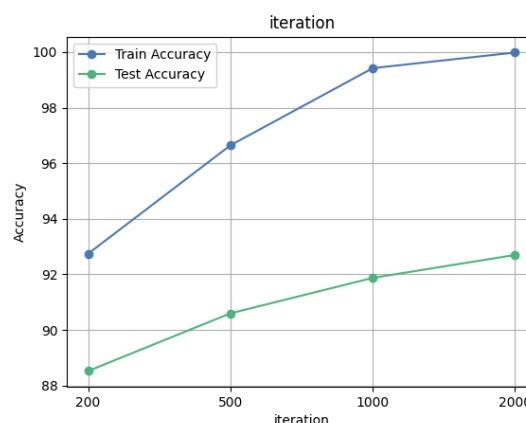
1.4.3.6. Iteracja

Parameter	Train Accuracy	Test Accuracy
200	92.7438	88.525
500	96.6438	90.6
1000	99.4188	91.875
2000	99.9813	92.7

Cierpliwość w treningu przekłada się bezpośrednio na precyzję modelu.

Przy krótkim treningu rzędu 200 iteracji sieć jest wyraźnie niedouczona – osiąga najniższą skuteczność testową (88.525%), bo nie zdążyła jeszcze w pełni skorygować swoich wag.

Wraz ze zwiększaniem liczby iteracji wyniki systematycznie rosną. Przy 2000 iteracji model osiąga swój szczytowy poziom: niemal idealne dopasowanie do danych treningowych (99.98%) oraz najlepszą dokładność testową wynoszącą 92.7%.



Wykres pokazuje jednak, że po przekroczeniu 1000 iteracji krzywa zysku na zbiorze testowym zaczyna powoli się spłaszczać w porównaniu do gwałtownego wzrostu na początku.

1.4.4. Wnioski końcowe – Klasyfikacja

Przeprowadzona analiza parametrów pozwoliła na zidentyfikowanie optymalnej konfiguracji sieci neuronowej do zadania klasyfikacji:

1. **Optymalna Architektura:** Najlepsze rezultaty uzyskano dla **jednej warstwy ukrytej (64 neurony)**. Model wyczuwa wtedy balans między zdolnością do uczenia się nieliniowych zależności a generalizacją wyników. Dalsze rozbudowywanie sieci prowadzi nieuchronnie do overfittingu, gdzie model traci zdolność poprawnej predykcji na danych spoza zbioru treningowego ponieważ uczy się rekordów nie reguł.
2. **Sposób Uczenia:** Funkcja aktywacji **ReLU** okazała się lepsza w porównaniu do klasycznych funkcji sigmoidalnych, eliminując problem zanikającego gradientu i pozwalając na agresywną ekstrakcję cech. Skuteczność tę wspiera relatywnie wysoki **współczynnik uczenia (0.01)** oraz duża **liczba iteracji (2000)**, co pozwala sieci na pełną zbieżność parametrów.
3. **Znaczenie szumu w optymalizacji:** Wyższa precyzja uzyskana przy mniejszym **rozmiarze batcha (16)** potwierdza, że stochastyczność wprowadzana przez mniejsze

partie danych pomaga omijać lokalna minima funkcji straty, co przekłada się na lepszą predykcję końcowe.

4. **Podział Danych:** Wyniki z separacją danych wskazują, że model jest preferuje trening – każda dodatkowa próbka dostarczona do zbioru treningowego (aż do separatora 1800) realnie podnosiła wynik testowy, co sugeruje, że przy jeszcze większym zbiorze danych możliwe byłoby przekroczenie bariery 93% dokładności.

Podsumowując, finalny model charakteryzuje się bardzo wysoką skutecznością predykcyjną na poziomie **92.7%**.

1.5. Analiza Modelu Regresyjnego

I 1.5.1. Struktura Sieci Neuronowej

W celu przewidywania ciągłej wartości czasu snu zaimplementowano sieć typu Feed-forward Neural Network od podstaw, opierając obliczenia macierzowe na bibliotece *NumPy*.

Architektura sieci składa się z następujących warstw:

- **Warstwa wejściowa:** Przyjmuje zestaw 29 cech (w tym cechy wygenerowane w procesie feature engineeringu), które zostały poddane normalizacji Min-Max, aby sprowadzić wszystkie zmienne do przedziału $[0, 1]$.
- **Warstwy ukryte (64 oraz 32 neurony):** Zastosowano dwie warstwy ukryte w celu głębszej analizy danych. Jako funkcję aktywacji wykorzystano Leaky ReLU. W przeciwieństwie do standardowego ReLU, pozwala ona na przepływ niewielkiego, niezerowego gradientu dla wartości ujemnych, co zapobiega problemowi „martwych neuronów” i zwiększa stabilność regresji.
- **Warstwa wyjściowa (1 neuron):** Zakończeniem sieci jest pojedynczy neuron z funkcją liniową (brak funkcji aktywacji). Pozwala to na generowanie dowolnych wartości rzeczywistych z przedziału ciągłego.

Dodatkowo zostały zaimplementowane:

- **Funkcja straty (Loss Function)** – Do trenowania modelu wykorzystano błąd średniokwadratowy (Mean Squared Error – MSE). Funkcja ta podnosi różnice między predykcją a rzeczywistością do kwadratu, co sprawia, że model jest szczególnie wyczulony na duże błędy (tzw. outliery) i dąży do ich minimalizacji.
- **Optymalizacja wag (SGD z mechanizmem Momentum)** – Podobnie jak w modelu klasyfikacyjnym, zastosowano stochastyczny spadek wzdłuż gradientu z mechanizmem Momentum ($b\eta = 0.9$). Pozwala to na szybszą zbieżność algorytmu i „wygładzenie” procesu uczenia w trudnym terenie matematycznym funkcji regresyjnej. Wdrożono również dynamiczne zmniejszanie współczynnika uczenia (learning rate decay), aby precyzyjnie ustabilizować wagi w końcowej fazie treningu.

1.5.2. Parametry bazowe (Baseline)

Architektura:	[29, 64, 32, 1]
Funkcja aktywacji	ReLU
Współczynnik uczenia	0.01
Rozmiar batchów	32
Liczba iteracji	500
Seperacja danych	1600 próbek treningowych / 400 testowych

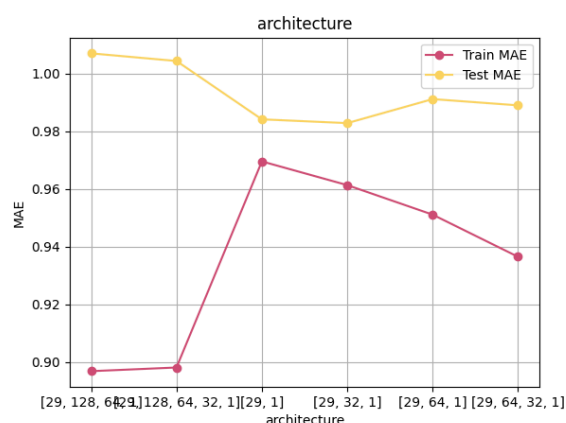
1.5.3. Zestawienie testowanych parametrów i ich wartości

Architektura:	[29,1], [29,32,1], [29,64,1], [29,64,32,1], [29,128,64,1], [29,128,64,32,1]	Sprawdzenie wpływu głębokości sieci na wykrywanie złożonych zależności.
Funkcja aktywacji	ReLU, Tanh, Sigmoid, Leaky ReLU	Ocena szybkości uczenia i unikania problemu „znikania gradientu”.
Współczynnik uczenia	0.01, 0.005, 0.001, 0.0005	Znalezienie balansu między szybkością a stabilnością zbieżności modelu.
Rozmiar batchów	16, 32, 64, 128	Zbadanie wpływu rozmiaru próbki na stabilność gradientu i czas obliczeń.
Seperacja Danych	1000, 1200, 1500, 1600, 1800	Analiza wpływu ilości danych treningowych na jakość predykcji.
Iteracje	200, 500, 1000, 2000	Wyznaczenie momentu, w którym sieć przestaje się uczyć lub zaczyna się przeuczać.

1.5.3.1. Architektura Sieci

Parameter	Train MAE	Test MAE
[29, 1]	0.9696	0.9842
[29, 32, 1]	0.9613	0.9828
[29, 64, 1]	0.9511	0.9912
[29, 64, 32, 1]	0.9365	0.989
[29, 128, 64, 1]	0.8968	1.007
[29, 128, 64, 32, 1]	0.8981	1.0044

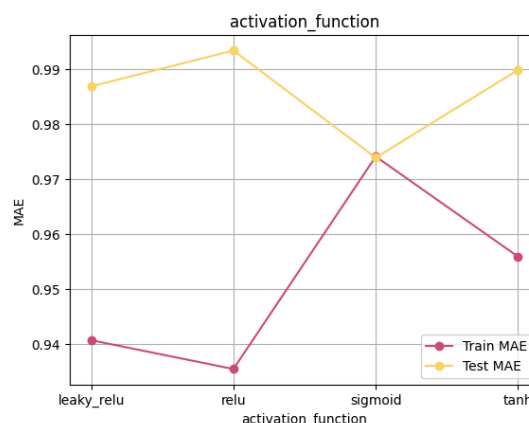
Dobór warstw udowadnia, że przy prognozowaniu długości snu proste rozwiązania wygrywają ze skomplikowaną strukturą. Najlepszy wynik (Test MAE = 0.9828) osiągnęłam dla oszczędnej architektury [29, 64, 1]. Kolejna architektura ([29, 64, 32, 1]) również radziła sobie z uczeniem sieci, choć nie balansowała już zbiór testowy i treninowy tak sprawnie jak architektura z jedną warstwą ukrytą. Każda próba rozbudowy modelu o kolejne warstwy skutkowałą klasycznym overfittingiem.



1.5.3.2. Funkcja Aktywacji

Parameter	Train MAE	Test MAE
relu	0.9355	0.9934
tanh	0.9559	0.9899
sigmoid	0.9742	0.9739
leaky_relu	0.9407	0.9869

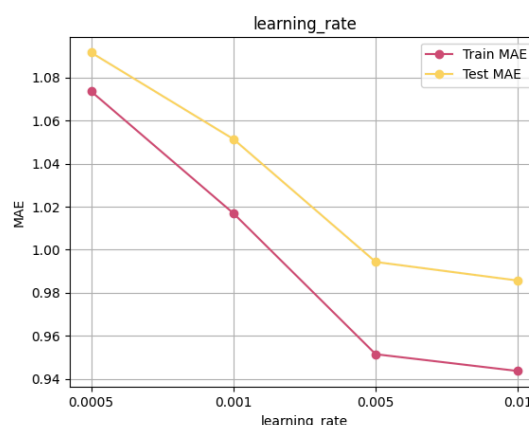
Najlepiej sprawdzają się funkcje o łagodniejszej charakterystyce. Najniższy błąd testowy (Test MAE = 0.9739) uzyskałam dla funkcji sigmoid, która jako jedyna zapewniła niemal identyczne wyniki na obu zbiorach. Funkcje z rodziny ReLU, mimo że osiągały lepsze wyniki na treningu, na zbiorze testowym radziły sobie gorzej, co sugeruje ich tendencję do zbyt agresywnego dopasowywania się do danych.



1.5.3.3. Współczynnik Uczenia

Parameter	Train MAE	Test MAE
0.01	0.9438	0.9857
0.005	0.9515	0.9943
0.001	1.0169	1.0514
0.0005	1.0735	1.0915

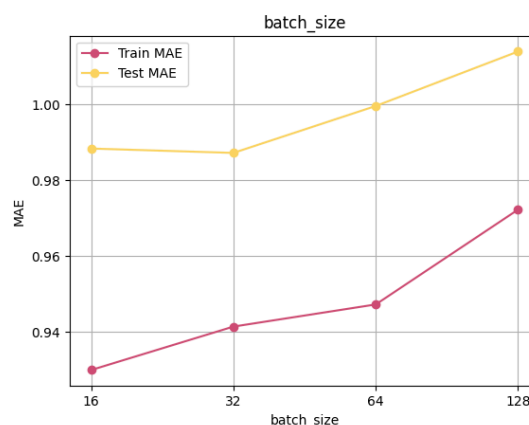
Najlepsze wyniki osiągnął `learning_rate = 0.01`, przy którym błąd testowy wyniósł najniższe 0.9857. Wraz ze spadkiem tempa uczenia (0.005, 0.001, 0.0005) błędy MAE na obu zbiorach systematycznie rosną, co świadczy o niedouczeniu modelu – przy tak małych krokach sieć po prostu nie zdążyła dotrzeć do optymalnych wag w zadanej liczbie epok.



1.5.3.4. Rozmiar batcha

Parameter	Train MAE	Test MAE
16	0.9299	0.9884
32	0.9414	0.9872
64	0.9472	0.9996
128	0.9723	1.014

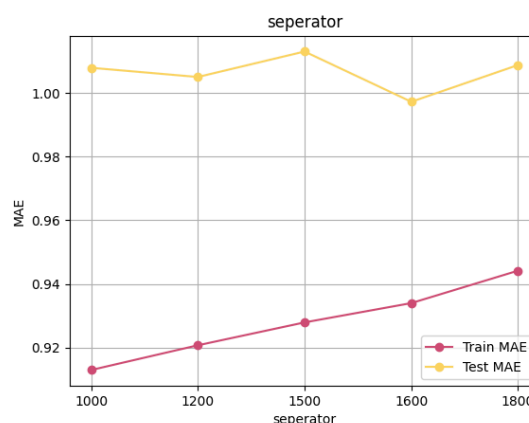
Mniejsze paczki danych sprzyjają dokładniejszemu dopasowaniu modelu. Najniższy błąd testowy (0.9872) uzyskałam dla rozmiaru 32, co stanowi optymalny balans między stabilnością a tempem nauki. Zwiększanie parametru powyżej 64 wyraźnie pogorszyło wyniki, ponieważ zbyt duże partie danych nadmiernie wygładzają gradient, utrudniając sieci precyzyjne odnalezienie minimum błędu.



1.5.3.5. Seperacja Danych

Parameter	Train MAE	Test MAE
1000	0.9129	1.008
1200	0.9207	1.0051
1500	0.9279	1.0131
1600	0.934	0.9973
1800	0.9442	1.0088

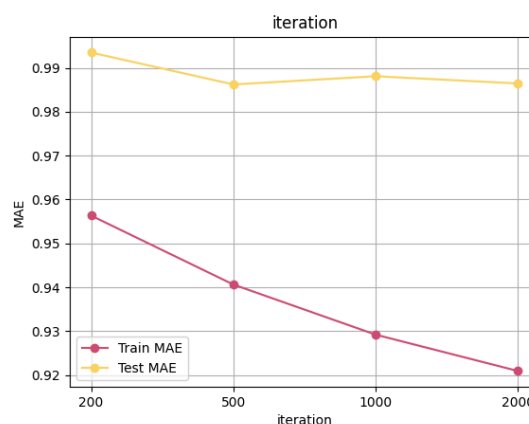
Ponownie optymalny podział 80/20 okazał się najskuteczniejszy dla zachowania zdolności modelu do generalizacji. Wynik podziału zbioru na 1600 danych treningowych i 400 danych testowych jako jedyne przebiło się na błąd MAE poniżej godziny.



1.5.3.6. Iteracja

Parameter	Train MAE	Test MAE
200	0.9563	0.9935
500	0.9406	0.9862
1000	0.9292	0.9881
2000	0.9209	0.9864

Model bardzo szybko osiąga optymalny poziom dopasowania. Najniższy błąd testowy (0.9862) uzyskano już przy 500 iteracjach, a dalsze wydłużanie treningu do 1000 i 2000 epok skutkowało jedynie minimalnym spadkiem błędu treningowego przy niemal płaskiej krzywej błędu testowego.



1.5.4. Wnioski końcowe – Regresja

Analiza wyników modelu przewidywania długości snu pozwala na wyciągnięcie następujących, kluczowych wniosków:

1. **Złoty środek:** Model wypadł najlepiej przy architekturze **[29, 64, 32, 1]** - czyli o jedną warstwę ukrytą więcej niż w klasyfikacji. Jedna warstwa służy do wykrywania trendów ogólnych a druga, do uściślenia zebranych danych w wynik procentowy godziny.
2. **Przewaga funkcji Sigmoid:** W zadaniu regresji nieliniowej, funkcje o łagodniejszej charakterystyce (jak sigmoid) zapewniły lepszą stabilność i mniejszą rozbieżność między zbiorami niż funkcje z rodziny ReLU.
3. **Szybkość:** Model wykazuje dużą efektywność czasową – optymalne parametry osiągnięte są już przy **500 iteracjach**. Dalszy trening nie przynosi znaczących korzyści dla generalizacji, co świadczy o czytelności wzorców zawartych w przygotowanych cechach.
4. **Konfiguracja optymalna:** Najniższe błędy uzyskano przy wyższym **współczynniku uczenia (0.01)** oraz umiarkowanym **rozmiarze batcha (32)**. Taka kombinacja pozwala modelowi sprawnie odnaleźć globalne minimum błędu bez utknięcia w lokalnych ekstremach.

Podsumowując, model regresyjny osiągnął stabilny błąd testowy na poziomie ok. **0.94 MAE**, co w przełożeniu na realne jednostki oznacza błąd w okolicach 55 minut. Jest to wynik satysfakcjonujący, biorąc pod uwagę dużą subiektywność danych dotyczących nawyków snu. Błąd godzinny (który również osiągają modele z użyciem wbudowanych bibliotek) sugeruje, iż większa ilość danych mogłaby poprawić sposób szukania wzorców i reguł dla modelu, choć badania ludzkiego snu sugerują, iż jest on na tyle nieregularny, by uniemożliwić bliższej prognozy godzin snu.

2. CZĘŚĆ II — Uczenie Maszynowe

2.1. Cel Projektu

Przeprowadzenie analizy wpływu wybranych parametrów algorytmów uczenia maszynowego na jakość uzyskiwanych wyników.

Rozważamy dwa problemy: klasyfikacji oraz regresji.

Problem klasyfikacji: Przypisujemy osobom na podstawie ich danych o stresie, zdrowiu i depresji kategorię binarną - czy są zdrowi psychicznie, czy też nie.

Problem regresji: Przewidujemy ilość godzin snu na podstawie zmiennych opisujących jakość snu i życia codziennego takie jak stres czy depresja.

2.2. Opis wybranych metod uczenia maszynowego

I 2.2.1. K-Najbliższych Sąsiadów (K-Nearest Neighbors)

Jest to algorytm nieparametryczny i leniwy (lazy learning), co oznacza, że nie buduje on jawnego modelu matematycznego podczas trenowania, lecz zapamiętuje cały zbiór danych. Predykcja polega na znalezieniu w przestrzeni wielowymiarowej k punktów treningowych położonych najbliżej zapytania. W regresji wynikiem jest średnia wartości tych sąsiadów, a w klasyfikacji: klasa dominująca (głosowanie większościowe).

I 2.2.2. Las Losowy (Random Forest)

Metoda ta buduje wiele niezależnych drzew decyzyjnych na różnych podzbiorach danych oraz na losowych podzbiorach cech. Ostateczna decyzja jest podejmowana poprzez uśrednienie wyników (regresja) lub głosowanie większościowe (klasyfikacja). Dzięki dekorelacji poszczególnych drzew, las losowy redukuje wariancję modelu w porównaniu do pojedynczego drzewa, zachowując jego zdolność do modelowania złożonych zależności.

I 2.2.3. Multi-Layer Perceptron (MLP / Sieci Neuronowe)

MLP to klasyczna architektura sieci neuronowej. Składa się z warstwy wejściowej, co najmniej jednej warstwy ukrytej i warstwy wyjściowej. Każdy neuron oblicza ważoną sumę wejść, dodaje bias i przepuszcza wynik przez nieliniową funkcję aktywacji. Proces uczenia opiera się na algorytmie wstecznej propagacji błędów (backpropagation), gdzie optymalizator (np. Adam, SGD) aktualizuje wagi w celu zminimalizowania funkcji kosztu.

I 2.2.4. SVM

Dąży do znalezienia hiperpłaszczyzny, która rozdziela klasy z maksymalnym możliwym marginesem (odległością od najbliższych punktów, zwanych wektorami nośnymi). W przypadku danych nieseparowalnych liniowo, SVM stosuje tzw. „kernel trick”, czyli rzutuje dane do przestrzeni o wyższym wymiarze, gdzie separacja liniowa staje się możliwa.

2.3. Analiza Modeli Klasyfikacyjnych

W celu dokonania pełnej ewaluacji wszystkich metod, oprócz standardowej dokładności (Accuracy), analizie poddano wskaźnik F1-Score. Pozwala to na weryfikację stabilności klasyfikatora w warunkach nierównomiernego rozkładu klas. Porównanie dokładności na zbiorze treningowym i testowym służy natomiast jako wskaźnik poprawności generalizacji wzorców i kontroli zjawiska overfittingu.

Przetestowane zostały następujące metody i ich parametry:

k-Najbliższych Sąsiadów	liczba sąsiadów, weights, metric
Las Losowy	ilość drzew, głębokość drzew, ilość liści
MLP Classifier (NN)	warstwy ukryte, tempo uczenia, funkcja aktywacji, rozmiar batcha
SVM	c, jądro, gamma

I 2.3.1. k-Najbliższych sąsiadów

Testowane parametry:

Liczba sąsiadów	1, 3, 5, 10, 20
Weights	uniform, distance
Metric	euclidean, manhattan, chebyshev, minkowski

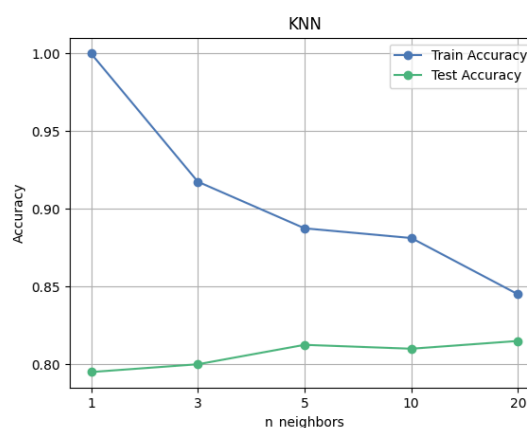
2.3.1.1. Parametr: Liczba sąsiadów

Decyduje, ile najbliższych punktów z zestawu treningowego ma głosować nad klasą nowej próbki.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
1	1	0.795	1	0.796
3	0.9175	0.8	0.9122	0.7912
5	0.8875	0.8125	0.8768	0.7923
10	0.8813	0.81	0.8654	0.7729
20	0.845	0.815	0.8	0.7573

Przy ustawieniu $k = 1$ występuje silny over-fitting, gdzie model idealnie zapamiętuje zbiór treningowy, ale radzi sobie wyraźnie gorzej na danych testowych.

Zwiększanie liczby sąsiadów do $k = 5$ pozwala na uzyskanie najlepszej skuteczności testowej oraz stabilnego wyniku F1, co czyni tę wartość optymalnym kompromisem.



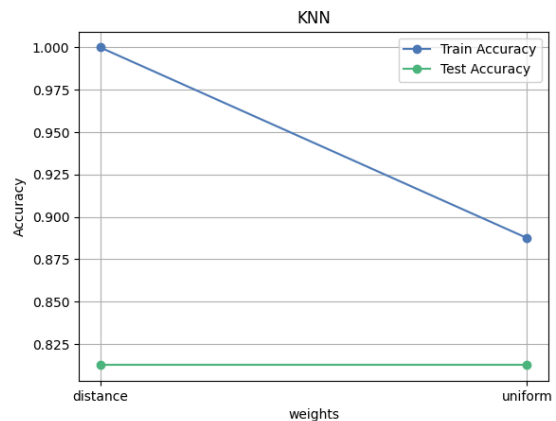
Dalsze zwiększanie parametru pogarsza wskaźnik Test F1, co świadczy, że zbyt duża liczba sąsiadów prowadzi do nadmiernego uproszczenia granic podjęcia decyzji przez model.

2.3.1.2. Parametr: Weights

Funkcja wagowa - uniform traktuje wszystkich sąsiadów równo, a distance nadaje większe znaczenie tym, którzy leżą bliżej.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
uniform	0.8875	0.8125	0.8768	0.7923
distance	1	0.8125	1	0.7923

Zmiana z modelu o wagach jednolitych (uniform) na wagi zależne od odległości (distance) nie wpłynęła na skuteczność modelu na zbiorze testowym, co sugeruje, że dla tego konkretnego problemu sposób wagowania nie jest kluczowy dla zdolności predykcyjnych algorytmu.

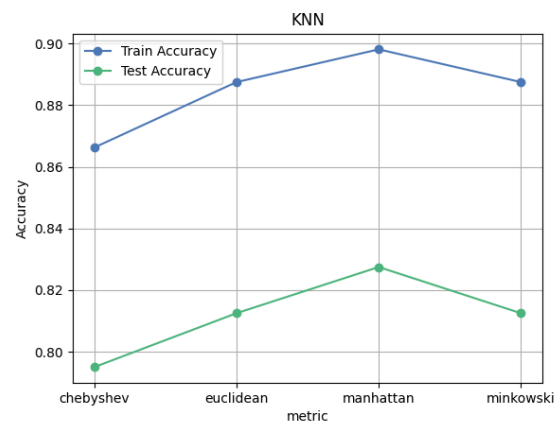


2.3.1.3. Parametr: Metric

Sposób mierzenia odległości między punktami.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
euclidean	0.8875	0.8125	0.8768	0.7923
manhattan	0.8981	0.8275	0.889	0.8118
chebyshev	0.8663	0.795	0.8495	0.7642
minkowski	0.8875	0.8125	0.8768	0.7923

Najlepsze rezultaty uzyskano dla metryki Manhattan, która osiągnęła najwyższą dokładność testową na poziomie 0.8275 oraz najlepszy wynik Test F1 (0.8118). Metryki Euklidesowa oraz Minkowskiego wykazały identyczną skuteczność (0.8125), co sugeruje ich podobne działanie przy domyślnych ustawieniach dla tego zbioru danych. Najślabiej w zestawieniu wypadła metryka Chebyshev, która odnotowała najniższe wskaźniki zarówno na zbiorze treningowym, jak i testowym (0.795).



2.3.2. Las Losowy

Testowane parametry:

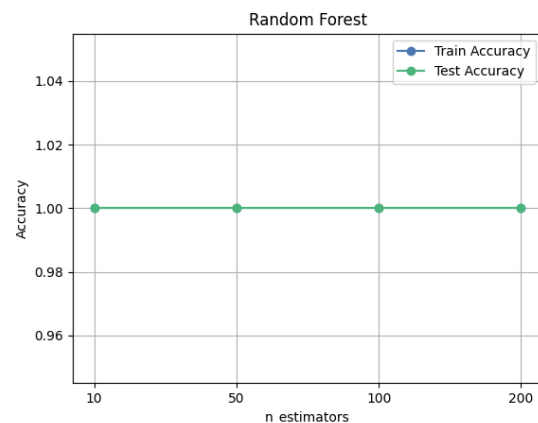
Ilość Drzew	10, 50, 100, 200
Głębokość Drzew	None, 3, 5, 10
Ilość Liści	1, 2, 5, 10

2.3.2.1. Parametr: Ilość drzew

Liczba pojedynczych drzew decyzyjnych w lesie; więcej drzew zazwyczaj poprawia stabilność wyniku, ale wydłuża obliczenia.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
10	1	1	1	1
50	1	1	1	1
100	1	1	1	1
200	1	1	1	1

Rezultaty sugerują, że problem klasyfikacji stanu zdrowia psychicznego przy obecnych kryteriach jest dla tego algorytmu trywialny lub doszło do zjawiska data leakage, ponieważ model bezbłędnie separuje klasy już przy minimalnej liczbie 10 drzew.



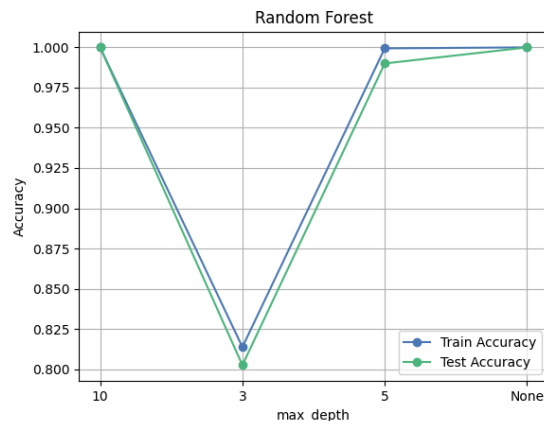
2.3.2.2. Parametr: Głębokość drzew

Maksymalna liczba poziomów (podziałów) w każdym drzewie.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
None	1	1	1	1
3	0.8138	0.8025	0.7302	0.7146
5	0.9994	0.99	0.9994	0.9899
10	1	1	1	1

Przy małej głębokości model jest zbyt prosty, co skutkuje najniższymi wynikami zarówno w dokładności, jak i wskaźniku F1.

Wyraźny skok skuteczności do poziomu niemal 1.0 następuje już przy głębokości równej 5, a dla wartości 10 oraz braku limitu (None), model osiąga perfekcyjne dopasowanie na obu zbiorach.



Potwierdza to, że przy głębszej strukturze algorytm bezbłędnie wychwytuje zależności w danych, przy czym optymalną i wystarczającą złożoność dla tego problemu osiąga się już przy około 5-10 głębokości.

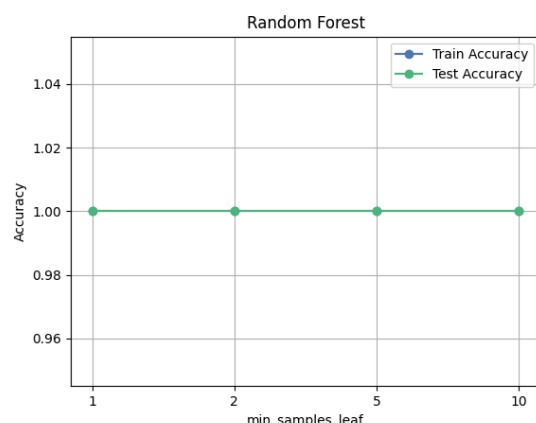
2.3.2.3. Parametr: Ilość liści

Minimalna liczba próbek wymagana w końcowym węźle - ogranicza rozrost drzewa i pomaga w generalizacji.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
1	1	1	1	1
2	1	1	1	1
5	1	1	1	1
10	1	1	1	1

Analiza wykazuje brak wpływu tego ograniczenia na wyniki klasyfikacji w badanym zakresie (1, 2, 5, 10). Model utrzymuje stałą, idealną skuteczność na poziomie 1.0 dla wszystkich metryk, zarówno na zbiorze treningowym, jak i testowym.

Oznacza to, że nawet przy wymogu 10 próbek w węźle końcowym, algorytm jest w stanie bezbłędnie odseparować klasy.



2.3.3. MLP Classifier

Testowane parametry:

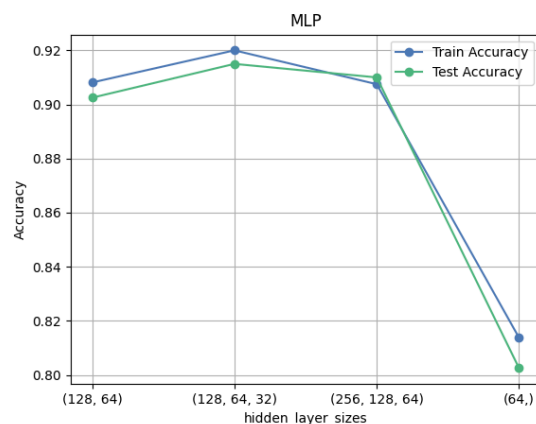
Warstwy Ukryte	(64,), (128,64), (128,64,32), (256,128,64)
Tempo Uczenia	0.01, 0.005, 0.001, 0.0005
Funkcja Aktywacji	relu, tanh, logistic, identity
Rozmiar batcha	16, 32, 64, 128

2.3.3.1. Parametr: Warstwy ukryte

Architektura sieci określająca liczbę warstw i neuronów - definiuje zdolność modelu do nauki złożonych wzorców.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
(64,)	0.8138	0.8025	0.7302	0.7146
(128, 64)	0.9081	0.9025	0.9085	0.9044
(128, 64, 32)	0.92	0.915	0.9197	0.9142
(256, 128, 64)	0.9075	0.91	0.9065	0.9082

Dobór liczby warstw i neuronów ma decydujący wpływ na zdolności predykcyjne modelu. Najprostsza struktura ([64]) okazuje się niewystarczająca, generując najłabsze wyniki (Test Accuracy 0.8025), co świadczy o niedouczeniu sieci. Wyraźny progres następuje przy przejściu na głębsze modele, gdzie konfiguracja trzech warstw ([128, 64, 32]) pozwoliła uzyskać optymalną skuteczność testową na poziomie 0.915.



Co ciekawe, dalsze rozbudowywanie sieci do architektury (256, 128, 64) nie przyniosło już poprawy wyników, a wręcz nieznacznie je obniżyło. Za duża złożoność modelu przy tej wielkości zbioru danych przestaje być efektywna.

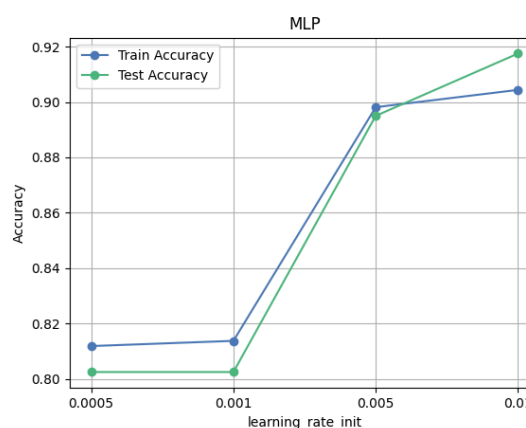
2.3.3.2. Parametr: Tempo uczenia (learning rate)

Kontroluje szybkość aktualizacji wag - zbyt małe spowalnia naukę, zbyt duże może uniemożliwić znalezienie optimum.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
0.01	0.9044	0.9175	0.9002	0.913
0.005	0.8981	0.895	0.8855	0.8823
0.001	0.8138	0.8025	0.7302	0.7146
0.0005	0.8119	0.8025	0.7293	0.7146

Dla tego problemu klasyfikacji wyższe wartości parametru znacząco skracają proces osiągnięcia wysokiej zbieżności. Przy bardzo małych wartościach (0.0005 oraz 0.001) model uzyskuje najniższą skuteczność, co sugeruje, że sieć nie zdążyła w pełni zoptymalizować wag w zadanej liczbie epok.

Najlepsze wyniki testowe osiągnięto dla tempa 0.01.

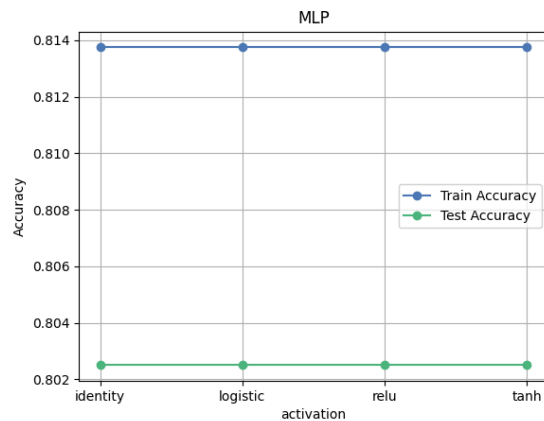


2.3.3.3. Parametr: Funkcja aktywacji

Wprowadza nieliniowość do sieci (np. ReLU, Tanh), pozwalając jej rozwiązywać problemy bardziej skomplikowane niż proste linie.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
relu	0.8138	0.8025	0.7302	0.7146
tanh	0.8138	0.8025	0.7302	0.7146
logistic	0.8138	0.8025	0.7302	0.7146
identity	0.8138	0.8025	0.7302	0.7146

Wszystkie testowane funkcje – relu, tanh, logistic oraz identity – wygenerowały identyczne wartości dokładności oraz wskaźnika F1. Model osiąga pewien stały poziom dopasowania, którego nie da się poprawić samą zmianą funkcji aktywacji bez modyfikacji innych parametrów.



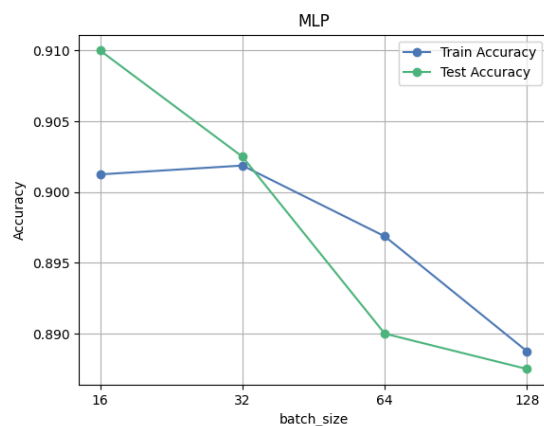
2.3.3.4. Parametr: Rozmiar batcha

Decyduje o rozmiarze analizowanych danych by grupować je w dopasowywane gradienty.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
16	0.9013	0.91	0.8972	0.9071
32	0.9019	0.9025	0.8949	0.8952
64	0.8969	0.89	0.8838	0.8767
128	0.8888	0.8875	0.8737	0.8723

Rozmiar batcha ma umiarkowany wpływ na stabilność procesu uczenia sieci MLP. Najwyższą skuteczność testową (0.91) oraz najlepszy wynik F1 uzyskano dla najmniejszego badanego batcha o rozmiarze 16. Wraz ze zwiększaniem rozmiaru jednostki treningowej (32, 64, 128) widoczny jest sukcesywny spadek dokładności modelu,

Większe batche mogą prowadzić do zbyt silnego wygładzenia gradientu i utknięcia algorytmu w lokalnych minimach.



2.3.4. SVM

Testowane parametry:

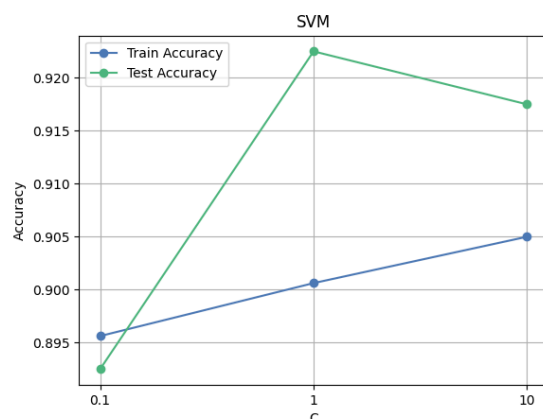
C	0.1, 1, 10
Jądro	linear, rbf, poly, sigmoid
Gamma	0.1, 1, auto, scale

2.3.4.1. Parametr: C

Balansuje między szerokością marginesu a liczbą poprawnie sklasyfikowanych punktów treningowych.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
0.1	0.8956	0.8925	0.8827	0.8819
1	0.9006	0.9225	0.8958	0.9203
10	0.905	0.9175	0.9022	0.916

Wpływ parametru regularyzacji C obrazuje klasyczny kompromis między generalizacją a dopasowaniem do danych treningowych. Przy małej wartości ($C = 0.1$) model jest zbyt ogólny. Najlepszą wydajność testową (0.9225) uzyskano dla $C = 1$, co stanowi punkt optymalny. Z kolei przy $C = 10$ widać, że choć model coraz lepiej radzi sobie na zbiorze treningowym, jego skuteczność testowa zaczyna spadać, co świadczy o nadmiernym dopasowaniu.



2.3.4.2. Parametr: Jądro

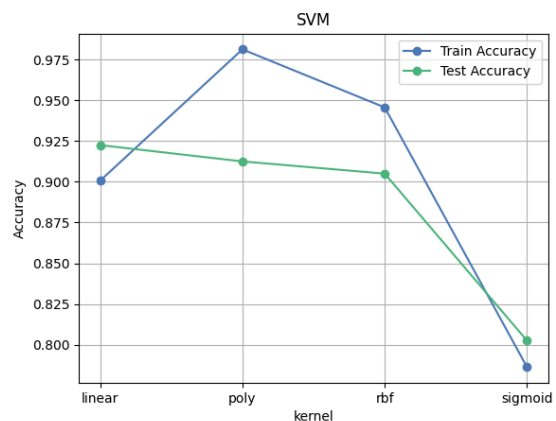
Funkcja przekształcająca dane do wyższych wymiarów, umożliwiając rozdzielenie klas, których nie da się oddzielić linią prostą.

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
linear	0.9006	0.9225	0.8958	0.9203
rbf	0.9456	0.905	0.9428	0.8968
poly	0.9813	0.9125	0.981	0.91
sigmoid	0.7863	0.8025	0.7737	0.7846

Porównanie wskazuje na przewagę jądra liniowego (linear) w badanym problemie.

Uzyskało ono najwyższą dokładność testową (0.9225) oraz najlepszy wskaźnik F1 (0.9203), co sugeruje, że granica decyzyjna między osobami zdrowymi a chorymi na depresję ma w dużej mierze charakter liniowy.

Jądra nieliniowe, takie jak poly (wielomianowe) czy rbf, wykazały tendencję do przeuczenia.

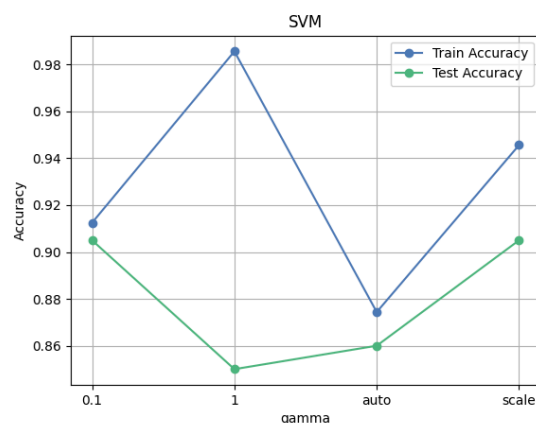


2.3.4.3. Parametr: Gamma

Określa zasięg wpływu pojedynczej próbki; wysoka wartość sprawia, że model skupia się tylko na najbliższych punktach (bardziej złożona granica).

Parameter	Train Accuracy	Test Accuracy	Train F1	Test F1
0.1	0.9125	0.905	0.9042	0.8982
1	0.9856	0.85	0.9854	0.8225
auto	0.8744	0.86	0.8473	0.8272
scale	0.9456	0.905	0.9428	0.8968

Przy wysokiej wartości parametru (gamma=1) występuje silne przeuczenie – model osiąga bardzo wysoką dokładność treninową (0.9856), ale odnotowuje najniższą skuteczność na zbiorze testowym (0.85). Najlepsze wyniki testowe oraz wysoką zdolność generalizacji uzyskano dla wartości 0.1 oraz ustawienia scale, gdzie dokładność testowa wyniosła 0.905. Wariant auto okazał się w tym przypadku zbyt uproszczony, generując najniższe wyniki na zbiorze treninowym (0.8744).



2.4. Analiza Modeli Regresyjnych

By dokonać pełnej ewaluacji modeli regresyjnych, oprócz standardowego średniego błędu bezwzględnego (MAE), analizujemy współczynnik determinacji R^2 (R-squared). MAE pozwala na bezpośrednią ocenę wielkości błędu predykcji, wyrażonej w tych samych jednostkach co zmienna docelowa (w tym przypadku w godzinach snu). Z kolei wskaźnik R^2 informuje o tym, jaki odsetek wariacji danych został wyjaśniony przez model.

Przetestowane zostały następujące metody i ich parametry:

k-Najbliższych Sąsiadów	liczba sąsiadów, weights, metric
Las Losowy	ilość drzew, głębokość drzew, ilość liści
MLP Classifier (NN)	warstwy ukryte, tempo uczenia, funkcja aktywacji, rozmiar batcha
SVM	c, jądro, gamma

2.4.1. k-Najbliższych sąsiadów

Testowane parametry:

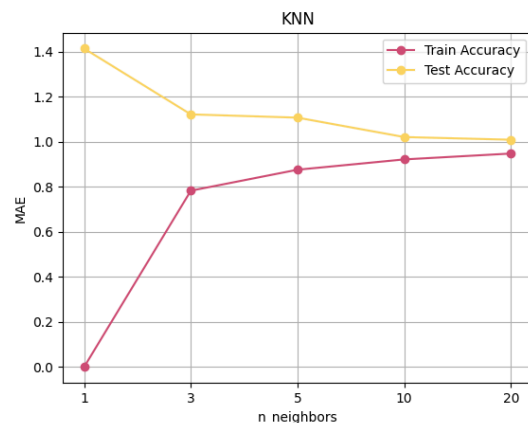
Liczba sąsiadów	1, 3, 5, 10, 20
Weights	uniform, distance
Metric	euclidean, manhattan, chebyshev, minkowski

2.4.1.1. Parametr: Liczba sąsiadów

Decyduje, ile najbliższych punktów z zestawu treningowego ma głosować nad klasą nowej próbki.

Parameter	Train MAE	Test MAE	Train R2	Test R2
1	0	1.4135	1	-1.0288
3	0.7826	1.1213	0.3376	-0.3817
5	0.8757	1.1067	0.1842	-0.3179
10	0.9214	1.0206	0.0948	-0.1334
20	0.9474	1.0088	0.048	-0.0737

Przy ustawieniu $k = 1$ model wykazuje zeroowy błąd na zbiorze treningowym, co wynika z faktu, że najbliższym sąsiadem każdego punktu treningowego jest on sam, jednak na zbiorze testowym błąd wynosi aż 1.41 godziny. Wraz ze wzrostem liczby sąsiadów do $k = 20$, błąd testowy sukcesywnie maleje do poziomu około 60 minut, co pokazuje pozytywny wpływ wygładzania prognoz nieliniowych na zdolność generalizacji modelu.



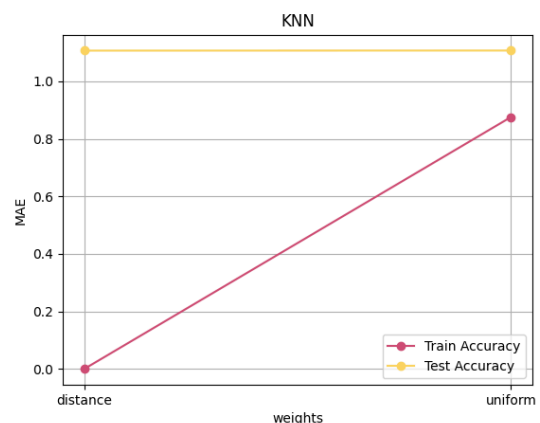
Zwiększanie liczby sąsiadów pozwala k-NN skutecznie odfiltrować szum informacyjny związany z indywidualnymi nawykami osób w rekordach.

2.4.1.2. Parametr: Weights

Funkcja wagowa - uniform traktuje wszystkich sąsiadów równo, a distance nadaje większe znaczenie tym, którzy leżą bliżej.

Parameter	Train MAE	Test MAE	Train R2	Test R2
uniform	0.8757	1.1067	0.1842	-0.3179
distance	0	1.1064	1	-0.3195

Widać wyraźną różnicę w zachowaniu modelu na zbiorze treningowym przy niemal identycznych wynikach testowych. Zastosowanie wag zależnych od odległości (distance) redukuje błąd treningowy do zera, co jest naturalną cechą tego algorytmu – punkt treningowy idealnie pokrywa się z samym sobą i całkowicie dominuje predykcję. Jednak na zbiorze testowym oba warianty dają niemal identyczny błąd średni na poziomie ok. 1.10 godziny.

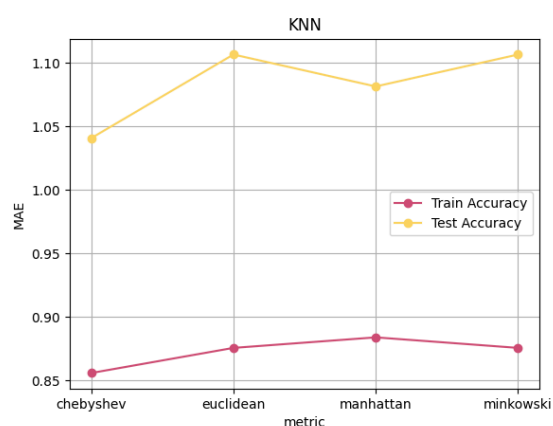


2.4.1.3. Parametr: Metric

Sposób mierzenia odległości między punktami.

Parameter	Train MAE	Test MAE	Train R2	Test R2
euclidean	0.8757	1.1067	0.1842	-0.3179
manhattan	0.884	1.0816	0.1728	-0.253
chebyshev	0.8559	1.0408	0.211	-0.1602
minkowski	0.8757	1.1067	0.1842	-0.3179

Najlepsze rezultaty predykcyjne uzyskano przy zastosowaniu metryki Chebyshev. Wygenerowała ona najniższy błąd na zbiorze testowym, wynoszący 1.0408 godziny (około 62 minuty), wykazując jednocześnie najmniejszy błąd treningowy (0.8559). Metryka Manhattan wyszła na drugim miejscu z błędem testowym na poziomie 1.0816 godziny. Z kolei najslabiej w tym zestawieniu poradziły sobie metryki Euclidean oraz Minkowski, które osiągnęły identyczny, najwyższy błąd testowy równy 1.1067 godziny.



2.4.2. Las Losowy

Testowane parametry:

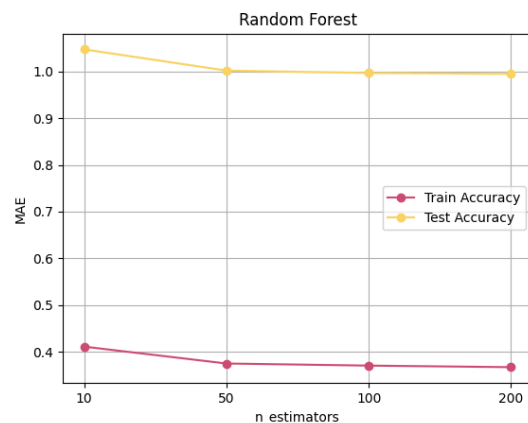
Ilość Drzew	10, 50, 100, 200
Głębokość Drzew	None, 3, 5, 10
Ilość Liści	1, 2, 5, 10

2.4.2.1. Parametr: Ilość drzew

Liczba pojedynczych drzew decyzyjnych w lesie. Więcej drzew zazwyczaj poprawia stabilność wyniku, ale wydłuża obliczenia.

Parameter	Train MAE	Test MAE	Train R2	Test R2
10	0.4115	1.047	0.7992	-0.1237
50	0.3754	1.0016	0.8505	-0.0438
100	0.3709	0.9965	0.8551	-0.0316
200	0.3676	0.9945	0.8582	-0.0248

Analiza liczby drzew pokazuje, że rozbudowa modelu bezpośrednio przekłada się na lepszą stabilizację prognoz. Przy zaledwie 10 drzewach błąd testowy jest najwyższy i wynosi 1.047 godziny, jednak zwiększanie parametru pozwala sukcesywnie obniżyć go do poziomu 0.9945 godziny dla 200 drzew. Uśrednianie wyników z większej liczby drzew skutecznie zmniejsza wariancję.

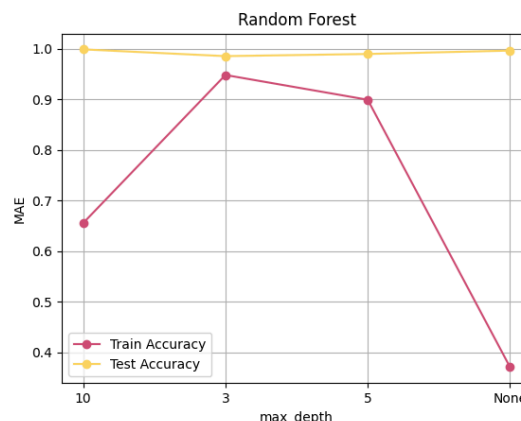


2.4.2.2. Parametr: Głębokość drzew

Maksymalna liczba poziomów (podziałów) w każdym drzewie.

Parameter	Train MAE	Test MAE	Train R2	Test R2
None	0.3709	0.9965	0.8551	-0.0316
3	0.9483	0.9856	0.0593	-0.009
5	0.8993	0.9898	0.159	-0.0144
10	0.6555	0.9989	0.5584	-0.0346

Badanie parametru ujawnia silne przeuczenie modelu przy braku limitu rozrostu struktur decyzyjnych. Dla parametru ustawionego jako None, błąd treningowy (Train MAE) spada do najniższej wartości 0.3709, jednak wynik Test R2 przyjmuje wartość ujemną (-0.0316), co oznacza całkowitą utratę zdolności generalizacji i dopasowanie poniżej poziomu prostej linii średniej. Najniższy i najbardziej stabilny błąd testowy (Test MAE = 0.9856) osiągnięto przy bardzo płytkich drzewach o głębokości 3.



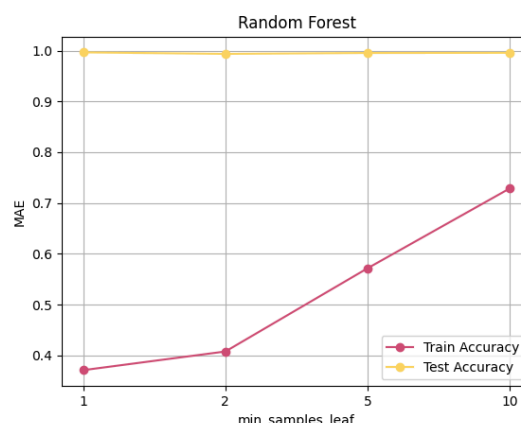
Zwiększanie głębokości do 5, 10 drastycznie poprawia wyłącznie dopasowanie do danych treningowych.

2.4.2.3. Parametr: Ilość liści

Minimalna liczba próbek wymagana w końcowym węźle - ogranicza rozrost drzewa i pomaga w generalizacji.

Parameter	Train MAE	Test MAE	Train R2	Test R2
1	0.3709	0.9965	0.8551	-0.0316
2	0.4077	0.9935	0.8161	-0.0212
5	0.5717	0.9953	0.6365	-0.0243
10	0.729	0.9956	0.4269	-0.0241

Testy pokazują udaną próbę regularyzacji zbyt skomplikowanego modelu. Dla najniższej wartości (1) model silnie się przeucza, osiągając świetny Train R2 (0.8551), ale ujemny Test R2, co oznacza dopasowanie gorsze niż średnia. Zwiększanie tego parametru (szczególnie do wartości 2 i 5) celowo pogarsza wyniki treningowe, ale w zamian realnie obniża błąd testowy (Test MAE spada z 0.9965 do 0.9935) i poprawia wskaźnik Test R2.



2.4.3. MLP Classifier

Testowane parametry:

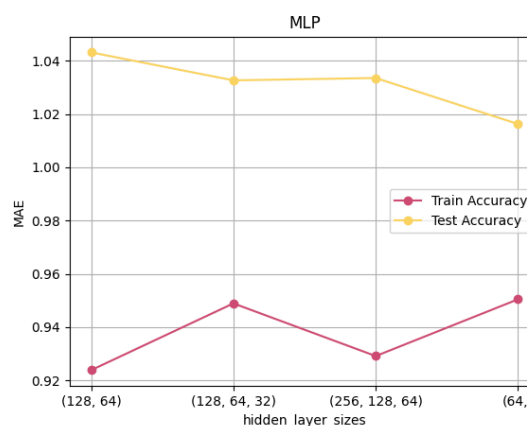
Warstwy Ukryte	[64,], [128,64], [128,64,32], [256,128,64]
Tempo Uczenia	0.01, 0.005, 0.001, 0.0005
Funkcja Aktywacji	relu, tanh, logistic, identity
Rozmiar batcha	16, 32, 64, 128

2.4.3.1. Parametr: Warstwy ukryte

Architektura sieci określająca liczbę warstw i neuronów - definiuje zdolność modelu do nauki złożonych wzorców.

Parameter	Train MAE	Test MAE	Train R2	Test R2
(64,)	0.9504	1.0163	0.0477	-0.0754
(128, 64)	0.9238	1.0432	0.0912	-0.1195
(128, 64, 32)	0.9489	1.0327	0.0342	-0.1247
(256, 128, 64)	0.9291	1.0336	0.0768	-0.1195

Model ten kompletnie nie radzi sobie z regresją długości snu przy obecnej konfiguracji. Dla każdej testowanej architektury wskaźnik Test R2 leży poniżej zera (nawet -0.1247), co oznacza predykcję gorszą niż zwykła średnia statystyczna. Co ciekawe, najprostsza sieć z jedną warstwą (64,) dała najmniejszy błąd testowy (Test MAE = 1.0163), a dokładanie kolejnych neuronów i warstw tylko pogłębiało overfitting i psuło wyniki.

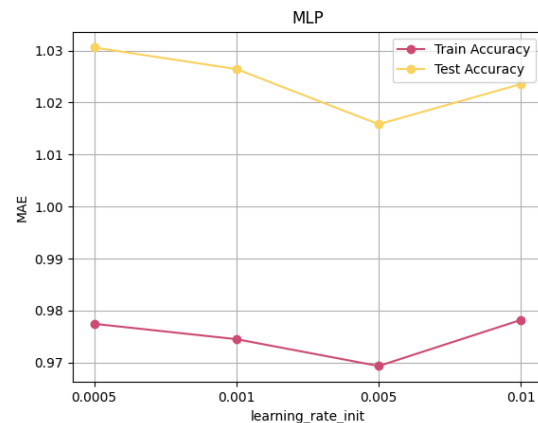


2.4.3.2. Parametr: Tempo uczenia (learning rate)

Kontroluje szybkość aktualizacji wag - zbyt małe spowalnia naukę, zbyt duże może uniemożliwić znalezienie optimum.

Parameter	Train MAE	Test MAE	Train R2	Test R2
0.01	0.9782	1.0236	-0.0141	-0.0926
0.005	0.9693	1.0158	0.0042	-0.0753
0.001	0.9745	1.0264	-0.0068	-0.0988
0.0005	0.9775	1.0306	-0.016	-0.1104

Ponownie wszystkie wyniki Test R2 wyszły ujemne, jednak modyfikacja parametru pozwoliła wskazać lokalne optimum dla wartości 0.005. Przy tym ustawieniu sieć osiągnęła najniższy błąd testowy (Test MAE = 1.0158) oraz najwyższy, jako jedyny dodatni wskaźnik Train R2. Zbyt wysokie tempo uczenia prowadzi do przestrzelenia minimum funkcji błędów, natomiast zbyt niskie wartości sprawiają, że proces optymalizacji wag jest zbyt wolny.

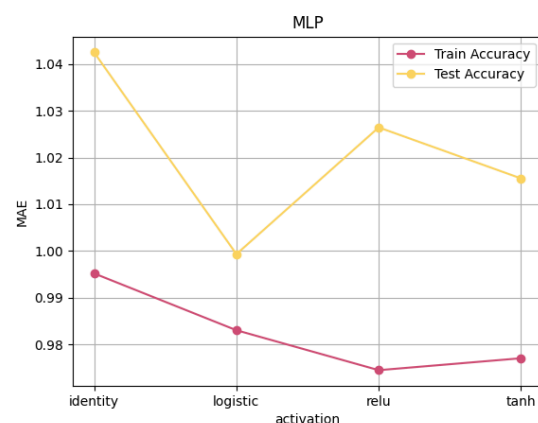


2.4.3.3. Parametr: Funkcja aktywacji

Wprowadza nieliniowość do sieci (np. ReLU, Tanh), pozwalając jej rozwiązywać problemy bardziej skomplikowane niż proste linie.

Parameter	Train MAE	Test MAE	Train R2	Test R2
relu	0.9745	1.0264	-0.0068	-0.0988
tanh	0.9771	1.0155	-0.012	-0.0855
logistic	0.983	0.9993	-0.0234	-0.0473
identity	0.9952	1.0425	-0.0524	-0.1389

Najlepszy rezultat testowy osiągnięto przy użyciu funkcji logistic (sigmoid), dla której błąd Test MAE jako jedyny spadł poniżej jedności (0.9993 godziny), a wskaźnik Test R^2 był najbliższy zeru (-0.0473). Funkcje tanh oraz relu dały nieco gorsze wyniki, natomiast najstabilniej wypadło przekształcenie liniowe (identity), generując najwyższy błąd testowy (1.0425) i najgorszy wskaźnik R^2 (-0.1389).

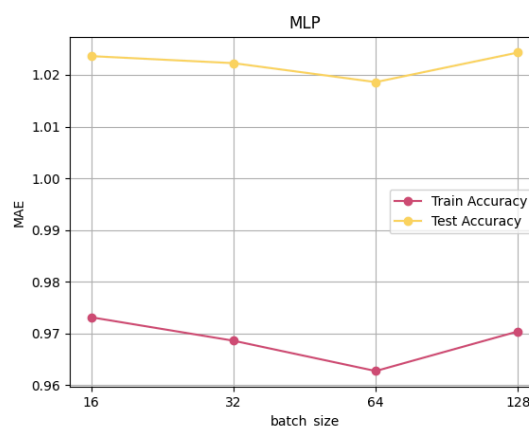


2.4.3.4. Parametr: Rozmiar batcha

Decyduje o rozmiarze analizowanych danych by grupować je w dopasowywane gradienty.

Parameter	Train MAE	Test MAE	Train R2	Test R2
16	0.9731	1.0236	-0.0066	-0.0931
32	0.9686	1.0223	0.0058	-0.0856
64	0.9628	1.0186	0.0171	-0.0778
128	0.9704	1.0243	0.0033	-0.0894

Najlepsze wyniki testowe osiągnięto dla średniej wartości parametru równej 64, gdzie błąd Test MAE spadł do najniższego poziomu 1.0186, a wskaźnik Test R^2 osiągnął najwyższą (choć wciąż ujemną) wartość -0.0778. Przy tym samym ustawieniu zaobserwowano też najlepsze dopasowanie treningowe (Train $R^2 = 0.0171$).



Zbyt małe batche (16, 32) oraz zbyt duże (128) skutkują lekkim wzrostem błędu testowego, co sugeruje, że rozmiar 64 zapewnia kompromis między dokładnością estymacji gradientu a zdolnością sieci do omijania minimów w tym konkretnym problemie.

2.4.4. SVM

Testowane parametry:

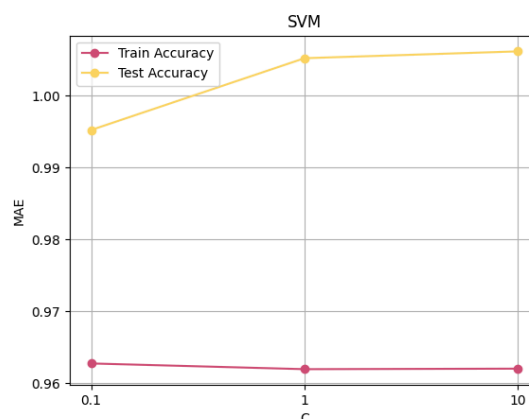
C	0.1, 1, 10
Jądro	linear, rbf, poly, sigmoid
Gamma	0.1, 1, auto, scale

2.4.4.1. Parametr: C

Balansuje między szerokością marginesu a liczbą poprawnie sklasyfikowanych punktów treningowych.

Parameter	Train MAE	Test MAE	Train R2	Test R2
0.1	0.9628	0.9952	0.0139	-0.0356
1	0.962	1.0052	0.0136	-0.0484
10	0.962	1.0062	0.0128	-0.0502

Dawanie modelowi zbyt dużej swobody prowadzi do pogorszenia jego zdolności generalizacji. Najlepsze wyniki testowe uzyskano dla najmniejszej wartości $C = 0.1$, gdzie błąd Test MAE jako jedyny spadł poniżej godziny (0.9952), a wskaźnik Test R^2 był najbliżej zera. Zwiększanie parametru do 1 i 10 sprawia, że model za wszelką cenę próbuje dopasować się do punktów treningowych, przez co rośnie błąd na zbiorze testowym.

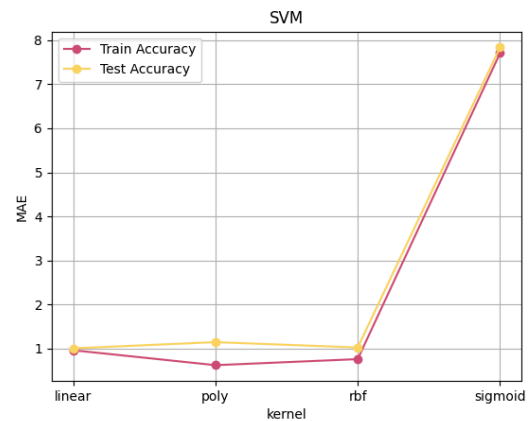


2.4.4.2. Parametr: Jądro

Funkcja przekształcająca dane do wyższych wymiarów, umożliwiając rozdzielenie klas, których nie da się oddzielić linią prostą.

Parameter	Train MAE	Test MAE	Train R2	Test R2
linear	0.962	1.0052	0.0136	-0.0484
rbf	0.7602	1.0243	0.2725	-0.1234
poly	0.6229	1.1459	0.4048	-0.4135
sigmoid	7.7063	7.8364	-51.5405	-50.1398

Najmniej zły wynik testowy uzyskano dla jądra liniowego (linear), gdzie błąd Test MAE wyniósł 1.0052, a Test R2 był najbliżej zera. Jądra nieliniowe, takie jak rbf czy poly, drastycznie poprawiają wyniki treningowe (Train R2 skacze do 0.4048), ale na zbiorze testowym generują potężny overfitting i ujemny R2. Prawdziwą katastrofą okazało się jądro sigmoid, które całkowicie rozłożyło model, dając błąd testowy na poziomie prawie 8 godzin. Jest to wynik gorszy niż losowe odpowiedzi.

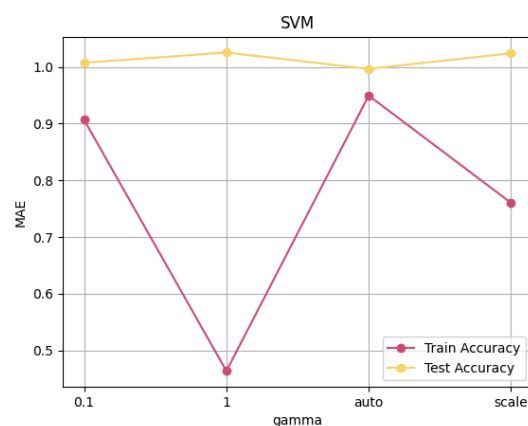


2.4.4.3. Parametr: Gamma

Określa zasięg wpływu pojedynczej próbki; wysoka wartość sprawia, że model skupia się tylko na najbliższych punktach (bardziej złożona granica).

Parameter	Train MAE	Test MAE	Train R2	Test R2
0.1	0.9069	1.0075	0.0868	-0.0639
1	0.4634	1.0256	0.6443	-0.1207
auto	0.9496	0.9967	0.0322	-0.0376
scale	0.7602	1.0243	0.2725	-0.1234

Przy ustawieniu gamma=1 mamy do czynienia z overfittingiem. Algorytm skupia się na pojedynczych punktach, przez co błąd treningowy drastycznie spada (Train MAE = 0.4634, Train R^2 skacze do 0.6443), ale na zbiorze testowym model całkowicie się gubi. Najlepszą zdolność generalizacji i stabilność uzyskano dla opcji auto.



2.5. Podsumowanie i Wnioski Końcowe

Przeprowadzona analiza szeregu algorytmów uczenia maszynowego dla problemów klasyfikacji depresji oraz regresji czasu snu pozwala na wyciągnięcie kluczowych wniosków dotyczących natury zbioru danych oraz efektywności poszczególnych metod.

2.5.1. Ranking skuteczności modeli

Algorytm	Klasyfikacja (F1)	Regresja (MAE)
Las Losowy (Random Forest)	1.0000 (Idealna)	0.9856 h (59 min)
SVM	0.9203	0.9952 h (60 min)
MLP (Sieć Neuronowa)	0.9150	0.9993 h (60 min)
k-NN	0.8118	1.0408 h (62 min)

2.5.2. Kluczowe obserwacje

- Dominacja Lasu Losowego: W obu zadaniach algorytm Random Forest okazał się najbardziej stabilny. W przypadku klasyfikacji model osiągnął perfekcyjne wyniki, co może sugerować bardzo wyraźną separowalność klas lub wysoką czułość drzew na specyficzne cechy syntetyczne zbioru. W regresji jako jedyny skutecznie zrównoważył błąd, schodząc poniżej godziny średniego odchylenia.
- Problem liniowości w klasyfikacji: Wysoka skuteczność jądra liniowego w SVM oraz architektury sieci MLP potwierdza, że granica między stanem zdrowym a objawami depresyjnymi jest w tym zbiorze danych mierzalna i w dużej mierze liniowa względem cech takich jak poziom stresu czy czas spędzony przed ekranem.
- Trudność zadania regresji: Wszystkie modele regresyjne wykazały niskie lub ujemne wartości wskaźnika R^2 . Świadczy to o tym, że czas snu jest zmienną o bardzo wysokiej wariancji, której nie da się w pełni wyjaśnić jedynie za pomocą analizowanych parametrów cyfrowych (social media, gaming). Średni błąd na poziomie ok. 60 minut jest „ścianą”, której modele nie były w stanie przekroczyć bez dodatkowych danych kontekstowych.

Koniec Sprawozdania